Contents

Course Outcomes (CO):

1. To develop ability to design the mechanism by which the performance of the system can be enhanced.
2. To understand memory storage & connections.
3. To understand the communication among processing elements inside the computer architecture.
4. To understand how to simulate the electronic circuits and measure their functionalities.

Program Specific Outcomes (PO)

PO NUMBER	SUMMARY	DESCRIPTION
PO1	Engineering knowledge	Apply theknowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of the complex engineering problems.
PO2	Problemanalysis	Identify, formulate, research literature, and analyze complex engineering problems reaching substantial conclusion using first principal of mathematics, natural science and Engineering sciences.
PO3	Design/ developmentof solutions	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and Environmental considerations.
PO4	Conduct investigations of complexproblems	Use research-based knowledge and research methods including designs of experiments analysis and interpretation of data and synthesis of information provide valid Conclusions.
PO5	Moderntool usage	Create, select, and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modeling to complex engineering values activities with an understanding of the limitation.
PO6	engineerand society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional Engineering practice.
PO7	Environmentand sustainability	Understand the impact to professional engineering solutions in societal and environmental contexts, and demonstrate the Knowledge of, and need for sustainable development.
PO8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9	Individualand teamwork	Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
P010	Communication	Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
P011	Project managementand finance	Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team,
P012	Life-long learning	Recognize the need for, and havethepreparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Outcomes (PSO)

PSO Number	Description
PSO1	Ability to understand the principles and development methodologies of computer systems. Students can assess the hardware of computer systems and possess professional skills and knowledge of software design process.
PSO2	Ability to apply mathematical methodologies to solve computation task,model real world problem using appropriate data structure And suitable algorithm.
PSO3	Ability to use knowledge in various domains to identify research Gaps and then provide solution to new ideas and innovations.

Program Educational Objectives (PEO)

PEO01: High Quality Engineering Design and Development Work:

Graduates of the program will engage in the effective practice of computer science and engineering to identify and solve important problems in adverse range of application areas.

PEO02: Real Life Problem Solving:

To educate students with proficiency in core areas of Computer science & Engineering and related engineering so as to comprehend engineering trade-offs, analyze, design, and synthesize data and technical concepts to create novel products and solutions for the real-life problems.

PEO03: Leadership:

Graduates of the program will engage in successful careers in industry, academia and attain positions of importance where they have impact to their business, profession and community.

PEO04: Life-long Learning:

Graduates of the program will adapt to contemporary technologies, tools and methodologies to remain at the frontier of computer science and engineering practice with the ability to respond to the need of a challenging environment.

Reference Resources

https://github.com/Abhiroop2004/Computer-Organization-and-Architecture-Lab

DEPARTMENT OF CSE (AIML)/ CSBS

Day 1: Familiarization of Hardware assembling for a digital computer

NAME OF THE STUDENT	
ROLL NO.	
DATE OF EXPERIMENT	
DATE OF SUBMISSION	
GRADE / MARKS	
TEACHER'S SIGNATURE	

Assignment 1: Familiarization of Hardware assembling for a digital computer

Theory:

Assembling a digital computer involves understanding various hardware components and their proper installation. Key components include the motherboard, processor (CPU), memory (RAM), storage devices (HDD/SSD), power supply unit (PSU), and peripheral devices. The process begins with preparing the workspace, ensuring all components are compatible, and carefully installing each part in the correct order. Proper handling of delicate components, cable management, and securing connections are essential to ensure optimal performance. Familiarization with these steps helps users confidently build, troubleshoot, and maintain a functional computer system.

Assignment 2: Familiarization of SPEC Benchmark Application for CPU

Theory:

The **Standard Performance Evaluation Corporation (SPEC) Benchmark** is a widely recognized tool used to measure and evaluate the performance of a CPU. SPEC benchmarks simulate real-world computing workloads to assess processing power, efficiency, and scalability. The tests include integer and floating-point operations, multi-threaded workloads, and power consumption analysis. Familiarizing with SPEC benchmarking involves installing the application, selecting appropriate test suites (e.g., SPEC CPU, SPECjbb, SPECint, SPECfp), executing tests, and analyzing performance metrics. This helps in comparing processors, optimizing system performance, and making informed hardware decisions.

DEPARTMENT OF CSE (AIML)/CSBS

Day 2: Realization of Boolean Expressions Using Basic Gates (IC Chips)

NAME OF THE STUDENT	
ROLL NO.	
DATE OF EXPERIMENT	
DATE OF SUBMISSION	
GRADE/MARKS	
TEACHER'S SIGNATURE	

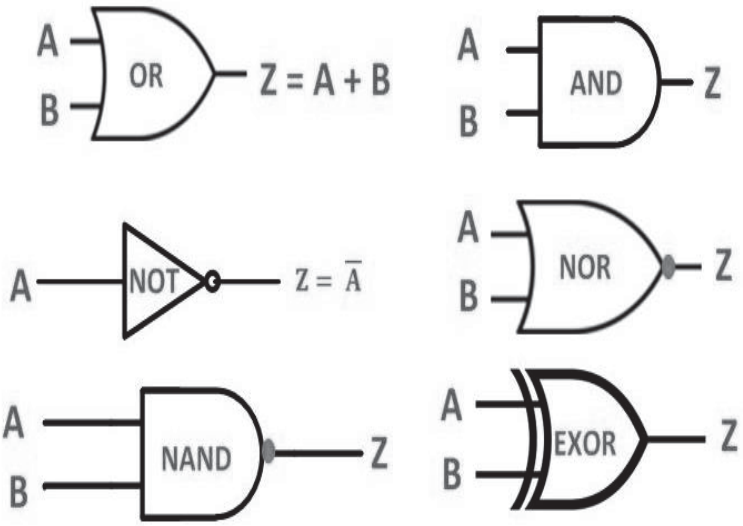
Assignment 1: Realization of Boolean Expressions Using Basic Gates (IC Chips)

Theory:

The realization of Boolean expressions using basic logic gates involves implementing logical functions using Integrated Circuit (IC) chips containing AND, OR, NOT, NAND, NOR, XOR, and XNOR gates. Each Boolean expression is simplified using algebraic methods like Boolean algebra or Karnaugh maps and then mapped to corresponding logic gate circuits. Standard ICs such as **7400 series (TTL) or 4000 series (CMOS)** contain these fundamental gates. By connecting inputs and outputs correctly, logical operations can be physically realized on a breadboard or PCB. This process is essential for designing combinational circuits, arithmetic units, and digital control systems.

IC Numbers List:

- AND Gate (7408)
- OR Gate (7432)
- NOT Gate (7404)
- NAND Gate (7400)
- NOR Gate (7402)
- XOR Gate (7486)
- XNOR Gate (74266)



Truth Table:

INPUT		OUTPUT				
A	B	AND	NAND	OR	NOR	XOR
0	0					
0	1					
1	0					
1	1					

INPUT	NOT GATE OUTPUT
0	
1	

DEPARTMENT OF CSE (AIML)/CSBS

Day 3: Design an 8:1 MUX unit using Basic Gates and using IC 74151

NAME OF THE STUDENT	
ROLL NO.	
DATE OF EXPERIMENT	
DATE OF SUBMISSION	
GRADE/MARKS	
TEACHER'S SIGNATURE	

Assignment 1: Design an 8:1 MUX unit using Basic Gates and using IC 74151

Objective:
To design and implement an 8-to-1 multiplexer using basic logic gates and the IC 74151, and to verify its functionality.

Implementation Using Basic Gates

Materials Required:

- Logic gates ICs: AND (7408), OR (7432), NOT (7404)
- Breadboard
- Connecting wires
- Power supply (+5V)
- LEDs for output indication
- Switches for input selection

Theory:
An 8-to-1 MUX selects one out of eight input signals based on the three select lines (S2, S1, S0). The output equation for an 8-to-1 multiplexer can be expressed as:

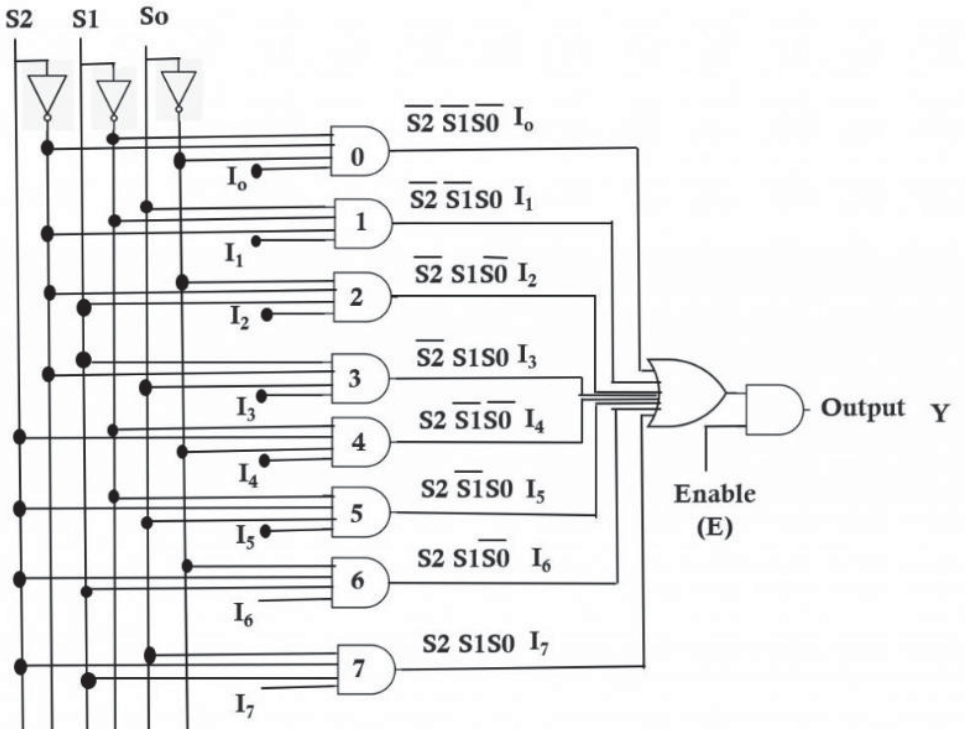
$$Y = (I_0 \cdot \overline{S_2} \cdot \overline{S_1} \cdot \overline{S_0}) + (I_1 \cdot \overline{S_2} \cdot \overline{S_1} \cdot S_0) + (I_2 \cdot \overline{S_2} \cdot S_1 \cdot \overline{S_0}) + (I_3 \cdot \overline{S_2} \cdot S_1 \cdot S_0) + (I_4 \cdot S_2 \cdot \overline{S_1} \cdot \overline{S_0}) + (I_5 \cdot S_2 \cdot \overline{S_1} \cdot S_0) + (I_6 \cdot S_2 \cdot S_1 \cdot \overline{S_0}) + (I_7 \cdot S_2 \cdot S_1 \cdot S_0)$$

Procedure:
1. Connect Power Supply: Connect V_{cc} (+5V) and GND to the ICs.
2. Implement NOT Gates: Use NOT gates (7404) to generate complements of select lines S2, S1, and S0.
3. Design AND Gates:
   Connect inputs (I0–I7) and select lines (S2, S1, S0) to eight AND gates following the expression above.
4. Use OR Gate: Connect outputs of all eight AND gates to a single OR gate (7432) to get the final MUX output.
5. Verify the Circuit: Apply different input combinations and verify the output with expected values.

Truth Table:

S2	S1	S0	Output (Y)
0	0	0	I0
0	0	1	I1
0	1	0	I2
0	1	1	I3
1	0	0	I4
1	0	1	I5
1	1	0	I6
1	1	1	I7

Circuit Diagram:



Implementation Using IC 74151

Materials Required:

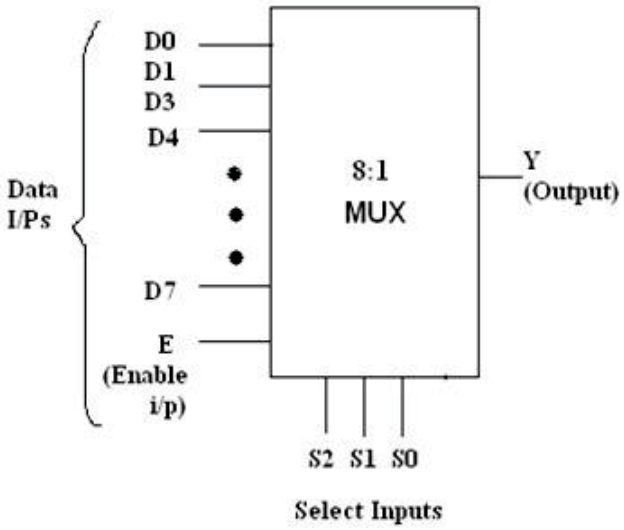
- IC 74151 (8-to-1 multiplexer)
- Breadboard
- Connecting wires
- Power supply (+5V)
- LEDs for output display
- Switches for inputs and select lines

Pin Configuration of IC 74151:

Pin	Function
1-7	Data Inputs (I0 to I7)
11, 10, 9	Select Lines (S2, S1, S0)
14	Output (Y)
15	Complemented Output (W)
6	Enable (Active Low)
16	V_{cc} (+5V)
8	GND

Procedure:

1. **Connect Power Supply:** Connect Vcc to Pin 16 and GND to Pin 8.
2. **Connect Select Lines:** Connect three select lines (S2, S1, S0) to switches or logic inputs.
3. **Provide Data Inputs:** Connect I0 to I7 to predefined logic levels (either HIGH or LOW).
4. **Enable the IC:** Ensure the Enable pin (Pin 6) is set to LOW for the MUX to function.
5. **Observe Output:** Check the output at Pin 14 (Y) by changing select line inputs.
6. **Verify Results:** Compare observed outputs with the expected truth table.



Enable	Select Inputs			Output
E	S2	S1	S0	Y
0	X	X	X	0
1	0	0	0	D0
1	0	0	1	D1
1	0	1	0	D2
1	0	1	1	D3
1	0	0	0	D4
1	0	0	1	D5
1	0	1	0	D6
1	0	1	1	D7

Observations:

- The circuit correctly selects one of the eight inputs based on the select lines.
- Using basic gates requires more components and wiring, whereas IC 74151 provides a compact and efficient solution.
- The observed outputs match the expected truth table values.

Conclusion:

An 8-to-1 multiplexer was successfully designed and implemented using both basic gates and IC 74151. The results validated the correct selection of input data based on select lines, demonstrating the working principle of multiplexers in digital circuit.

DEPARTMENT OF CSE(AIML)/CSBS

Day 4: Design of a 4-Bit Parallel Binary Adder Circuit Using the IC-Chip7483

NAME OF THE STUDENT	
ROLL NO.	
DATE OF EXPERIMENT	
DATE OF SUBMISSION	
GRADE/MARKS	
TEACHER'S SIGNATURE	

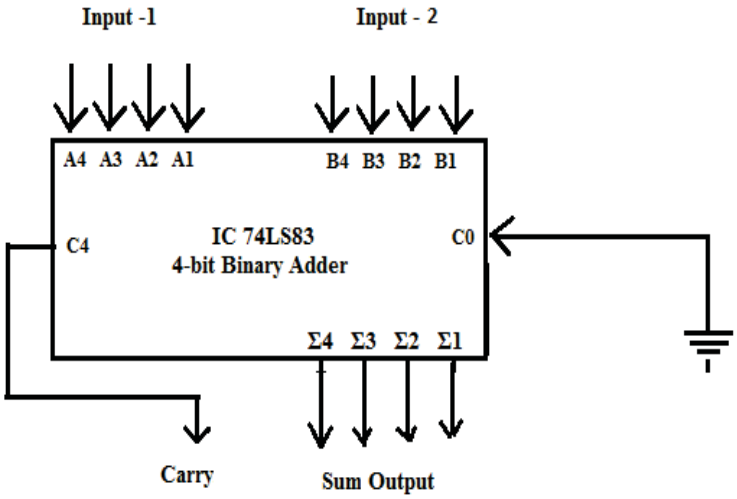
Assignment 1: Design of a 4-Bit Parallel Binary Adder Circuit Using The IC-Chip7483

Aim:To implement a 4-bit binary parallel adder using the IC 7483 and verify its functionality.

IC Used: IC 7483 (4-bit Full Adder)

Pin Configuration:

- Inputs: A0-A3, B0-B3
- Carry In: Pin 9 (Cin)
- Sum Outputs: S0-S3
- Carry Out: Pin 14 (Cout)



Procedure:

1. Connect inputs A0-A3 and B0-B3 to binary values.
2. Connect Cin to 0 for addition without carry or 1 for carry addition.
3. Observe S0-S3 for sum outputs and Cout for the carry out.

Truth Table:

A3 A2 A1 A0	B3 B2 B1 B0	Cin	S3 S2 S1 S0	Cout
0000	0000	0	0000	0
0001	0001	0	0010	0
1111	0001	0	0000	1
1010	0101	0	1111	0

Observations:

- Verify the sum and carry output for each input combination.

Conclusion:
The 4-bit binary parallel adder using IC 7483 was successfully implemented and verified. The outputs matched the expected results.

DEPARTMENT OF CSE(AIML)/CSBS

Day 5: Introduction of VHDL and Verilog programming language

NAME OF THE STUDENT	
ROLL NO.	
DATE OF EXPERIMENT	
DATE OF SUBMISSION	
GRADE/MARKS	
TEACHER'S SIGNATURE	

Assignment 1: Introduction of VHDL and Verilog programming language

Theory: VHDL and Verilog are **Hardware Description Languages (HDLs)** used for designing and modeling digital circuits. These languages enable designers to describe the behavior, structure, and implementation of hardware systems at various abstraction levels, such as **behavioral**, **dataflow** and **structural** modeling. They are widely used in **FPGA (Field-Programmable Gate Array)** and **ASIC (Application-Specific Integrated Circuit)** design.

VHDL (Very High-Speed Integrated Circuit Hardware Description Language)

Overview:
VHDL is a strongly typed, structured HDL developed by the U.S. Department of Defense and standardized by IEEE (IEEE 1076). It is commonly used in industry and academia for designing digital circuits.

Key Features:
- **Strongly Typed Language:** Requires explicit data types, ensuring high reliability.
- **Supports Concurrency:** Allows modeling of parallel digital circuits efficiently.
- **Modular Design:** Encourages reusable code through libraries and packages.
- **Multiple Design Abstractions:** Supports behavioral, dataflow, and structural modeling.

Verilog

Overview:
Verilog is a widely used HDL developed in the 1980s by Gateway Design Automation. It is more C-like, making it easier to learn for programmers familiar with C or Java. It is also standardized by IEEE (IEEE 1364).

Key Features:
- **Simple Syntax:** Similar to C, making it easier to write and debug.
- **Event-Driven Simulation:** Uses always blocks to describe circuit behavior.
- **Supports Both Low-Level and High-Level Modeling:** Used for both RTL (Register Transfer Level) and gate-level designs.
- **Efficient for Synthesis:** Frequently used in FPGA and ASIC implementations

Feature	VHDL	Verilog
Syntax	Strongly typed, verbose	C-like, concise
Data Types	Rich data types (arrays, records)	Fewer data types
Learning Curve	Steeper	Easier for C programmers
Usage	Preferred in Europe, defense, and safety-critical applications	Widely used in the U.S. and industry
Simulation	Slower but more accurate	Faster and easier for debugging

Types of VHDL Models

VHDL provides multiple modeling styles to describe digital circuits at different levels of abstraction. The primary types of VHDL models are:

- Dataflow Modeling
- Behavioral Modeling
- Structural Modeling

Dataflow Modeling

- Describes circuit functionality using concurrent statements.
- Uses signal assignment (<=) and logical operators.
- More efficient than behavioral modeling for simple combinational logic.

Behavioral Modeling

- Describes what a circuit does rather than its internal structure.
- Uses sequential statements (e.g., if-else, case, loop) inside process blocks.
- Ideal for high-level simulation and algorithm implementation.

Structural Modeling

- Describes a circuit in terms of its components and interconnections.
- Uses component instantiation to connect different modules.
- Best suited for complex designs like processors and large digital systems.

Assignment 2: Implementation of basic gates using VHDL and Verilog (Dataflow Model)

AND GATE:

Dataflow Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity AND_Gate_Dataflow is
   Port (
      A, B: in  STD_LOGIC;
      Y: out STD_LOGIC
   );
end AND_Gate_Dataflow;

architecture Dataflow of
AND_Gate_Dataflow is
begin
   Y <= A and B; -- Direct assignment
(dataflow)
end Dataflow;
```

Behavioral Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity AND_Gate_Behavioral is
   Port (
      A, B: in  STD_LOGIC;
      Y: out STD_LOGIC
   );
end AND_Gate_Behavioral;

architecture Behavioral of
AND_Gate_Behavioral is
begin
   process(A, B)
   begin
      if (A = '1' and B = '1') then
         Y <= '1';
      else
         Y <= '0';
      end if;
   end process;
end Behavioral;
```

RTL schematic for AND gate

OR GATE:

Dataflow Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity OR_Gate_Dataflow is
   Port (
      A, B : in  STD_LOGIC;
      Y    : out STD_LOGIC
   );
end OR_Gate_Dataflow;

architecture Dataflow of OR_Gate_Dataflow is
begin
   Y <= A or B; -- Direct assignment (dataflow)
end Dataflow;
```

Behavioral Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity OR_Gate_Behavioral is
   Port (
      A, B: in  STD_LOGIC;
      Y: out STD_LOGIC
   );
end OR_Gate_Behavioral;

architecture Behavioral of OR_Gate_Behavioral
is
begin
   process(A, B)
   begin
      if (A = '1' or B = '1') then
         Y <= '1';
      else
         Y <= '0';
      end if;
   end process;
end Behavioral;
```

RTL schematic for OR gate

NAND GATE:

Dataflow Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NAND_Gate_Dataflow is
   Port (
      A, B : in  STD_LOGIC;
      Y    : out STD_LOGIC
   );
end NAND_Gate_Dataflow;

architecture Dataflow of
NAND_Gate_Dataflow is
begin
   Y <= A nand B; -- Direct NAND operation
end Dataflow;
```

Behavioral Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NAND_Gate_Behavioral is
   Port (
      A, B : in  STD_LOGIC;
      Y    : out STD_LOGIC
   );
end NAND_Gate_Behavioral;

architecture Behavioral of
NAND_Gate_Behavioral is
begin
   process(A, B)
   begin
      if (A = '1' and B = '1') then
         Y <= '0';
      else
         Y <= '1';
      end if;
   end process;
end Behavioral;
```

RTL schematic for NAND gate

NOR GATE:

Dataflow Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NOR_Gate_Dataflow is
   Port (
      A, B: in  STD_LOGIC;
      Y: out STD_LOGIC
   );
end NOR_Gate_Dataflow;

architecture Dataflow of NOR_Gate_Dataflow
is
begin
   Y <= A nor B; -- Direct NOR operation
end Dataflow;
```

Behavioral Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NOR_Gate_Behavioral is
   Port (
      A, B: in  STD_LOGIC;
      Y: out STD_LOGIC
   );
end NOR_Gate_Behavioral;

architecture Behavioral of
NOR_Gate_Behavioral is
begin
   process(A, B)
   begin
      if (A = '0' and B = '0') then  -- NOR
condition
         Y <= '1';
      else
         Y <= '0';
      end if;
   end process;
end Behavioral;
```

RTL schematic for NOR gate

XOR GATE:

Dataflow Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity XOR_Gate_Dataflow is
   Port (
      A, B : in  STD_LOGIC;
      Y    : out STD_LOGIC
   );
end XOR_Gate_Dataflow;

architecture Dataflow of XOR_Gate_Dataflow
is
begin
   Y <= A xor B; -- Direct XOR operation
end Dataflow;
```

Behavioral Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity XOR_Gate_Behavioral is
   Port (
      A, B : in  STD_LOGIC;
      Y    : out STD_LOGIC
   );
end XOR_Gate_Behavioral;

architecture Behavioral of
XOR_Gate_Behavioral is
begin
   process(A, B)
   begin
      if (A /= B) then  -- XOR condition
         Y <= '1';
      else
         Y <= '0';
      end if;
   end process;
end Behavioral;
```

RTL schematic for XOR gate

NOT GATE:

Dataflow Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NOT_Gate_Dataflow is
   Port (
      A : in  STD_LOGIC;
      Y : out STD_LOGIC
   );
end NOT_Gate_Dataflow;

architecture Dataflow of NOT_Gate_Dataflow
is
begin
   Y <= not A; -- Direct NOT operation
end Dataflow;
```

Behavioral Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NOT_Gate_Behavioral is
   Port (
      A : in  STD_LOGIC;
      Y : out STD_LOGIC
   );
end NOT_Gate_Behavioral;

architecture Behavioral of
NOT_Gate_Behavioral is
begin
   process(A)
   begin
      if (A = '1') then
         Y <= '0';
      else
         Y <= '1';
      end if;
   end process;
end Behavioral;
```

RTL schematic for NOT gate

:

Waveforms of all Logic Gates:

DEPARTMENT OF CSE (AIML)/CSBS

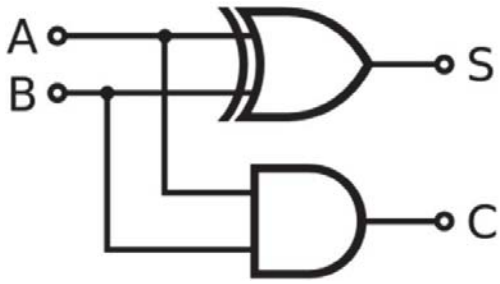
Day 6: Implementation of Half and Full Adder, Half and Full Sub-tractor using VHDL (Dataflow& Behavioral Model)

NAME OF THE STUDENT	
ROLL NO.	
DATE OF EXPERIMENT	
DATE OF SUBMISSION	
GRADE/MARKS	
TEACHER'S SIGNATURE	

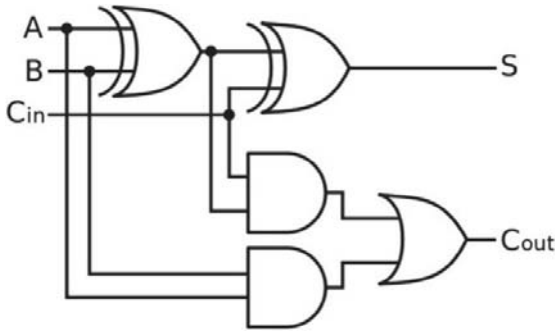
Theory:

Half-Adder

A half-adder is a combinational logic circuit that performs the addition of two bits, A and B. The output of the half-adder is two bits: the sum bit, S, and the carry bit, C. The Boolean functions describing the half-adder are as follows: **Sum bit: S = A ⊕ B&Carry bit: C = A·B**



Half-Adder using X-OR and basic gates



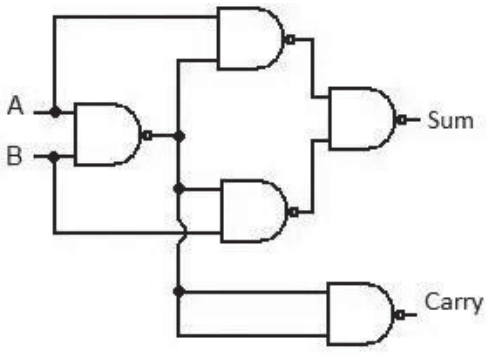
Half-Adder using NAND gate only

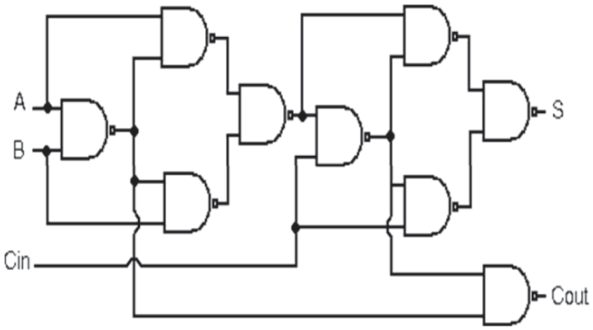
Full-Adder

A full-adder is a combinational logic circuit that performs the addition of two bits, A and B, and a carry-in bit, Cin. The output of the full-adder is two bits: the sum bit, S, and the carry bit, Cout.

Sum bit: $S = (A \oplus B) \oplus C_{in}$
Carry bit: $C_{out}=A\cdot B+C_{in}\cdot(A\oplus B)$ Full-Adder using X-OR and basic-gates

Full-Adder using NAND gate only





Truth Table

Half Adder

INPUT		OUTPUT	
A	**B**	**Sum**	**Carry**
0	0		
0	1		
1	0		
1	1		

K-map: Sum

A \ B	B'(0)	B(1)
A'(0)		
A(1)		

Sum:

Carry

A \ B	B'(0)	B (1)
A' (0)		
A (1)		

Carry:

Full Adder

INPUT			OUTPUT	
A	**B**	**C_{in}**	**Sum**	**C_{out}**
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

K-map: Sum

BC_{in} \ A	$B'C_{in}'$ (00)	$B'C_{in}$(01)	BC_{in}(11)	BC_{in}' (10)
A'(0)				
A(1)				

Sum:

Carry

BC_{in} \ X	$B'C_{in}'$ (00)	$B'C_{in}$(01)	BC_{in}(11)	BC_{in}' (10)
A'(0)				
A(1)				

Carry:

Assignment 1: Implementation of Half and Full Adder, using VHDL (Dataflow Model)
Assignment 2: Implementation of Half and Full Adder, using VHDL (Behavioral Model)
Assignment 3: Implementation of Half and Full Subtractor, using VHDL (Dataflow Model)
Assignment 4: Implementation of Half and Full Subtractor, using VHDL (Behavioral Model)

Half Adder

Dataflow Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HalfAdder_Dataflow is
   Port (
      A, B    : in  STD_LOGIC;
      Sum, Cout : out STD_LOGIC
   );
end HalfAdder_Dataflow;

architecture Dataflow of
HalfAdder_Dataflow is
begin
   Sum  <= A xor B;  -- Sum output
   Cout <= A and B;  -- Carry output
end Dataflow;
```

Behavioral Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HalfAdder_Behavioral is
   Port (
      A, B    : in  STD_LOGIC;
      Sum, Cout : out STD_LOGIC
   );
end HalfAdder_Behavioral;

architecture Behavioral of HalfAdder_Behavioral is
begin
   process(A, B)
   begin
      -- Sum calculation
      if (A /= B) then
         Sum <= '1';
      else
         Sum <= '0';
      end if;
      -- Carry calculation
      if (A = '1' and B = '1') then
         Cout <= '1';
      else
         Cout <= '0';
      end if;
   end process;
end Behavioral;
```

Full Adder

Dataflow Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FullAdder_Dataflow is
   Port (
      A, B, Cin : in  STD_LOGIC;
      Sum, Cout : out STD_LOGIC
   );
end FullAdder_Dataflow;

architecture Dataflow of
FullAdder_Dataflow is
begin
   Sum  <= A xor B xor Cin;
   Cout <= (A and B) or (Cin and (A xor B));
end Dataflow;
```

Behavioral Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FullAdder_Behavioral is
   Port (
      A, B, Cin : in  STD_LOGIC;
      Sum, Cout : out STD_LOGIC
   );
end FullAdder_Behavioral;

architecture Behavioral of FullAdder_Behavioral is
begin
   process(A, B, Cin)
      variable temp_sum : STD_LOGIC;
   begin
      temp_sum := A xor B;
      Sum <= temp_sum xor Cin;

      if ((A and B) or (Cin and temp_sum)) = '1' then
         Cout <= '1';
      else
         Cout <= '0';
      end if;
   end process;
end Behavioral;
```

RTL Schematics:

Waveforms:

Half Subtractor

Dataflow Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity half_sub is
port( A, B : in std_logic;
DIFF, Borrow : out std_logic);
end entity;

Architecture dataflow of half_sub is
begin

DIFF <= A xor B;
Borrow <= (not A) and B;



end architecture;
```

Behavioral Model

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.STD_LOGIC_ARITH.ALL;

use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity HALFSUBTRACTOR_BEHAVIORAL_SOURCE is
   Port ( A : in  STD_LOGIC_VECTOR (1 downto 0);
          Y : out  STD_LOGIC_VECTOR (1 downto 0));
end HALFSUBTRACTOR_BEHAVIORAL_SOURCE;
architecture Behavioral of
HALFSUBTRACTOR_BEHAVIORAL_SOURCE is
begin
process(A)
begin
if (A = "00" or A = "11") then
Y<="00";
else if (A = "01") then

Y<="11";
Else
Y<="10";
end if;
end if;
end process;
end Behavioral;
```

RTL Schematics:

Full Sub-tractor

Dataflow Model

```
library IEEE;
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity full_sub is
port( A, B, C : in std_logic;
DIFF, Borrow : out std_logic);
end entity;

architecture dataflow of full_sub is
begin

DIFF <= (A xor B) xor C;
Borrow <= ((not A) and (B or C)) or (B and
C);


end dataflow;
```

Behavioral Model

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.STD_LOGIC_ARITH.ALL;

use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity FULLSUBTRACTOR_BEHAVIORAL_SOURCE is

Port ( A : in  STD_LOGIC_VECTOR (2 downto 0);

Y : out  STD_LOGIC_VECTOR (1 downto 0));
end FULLSUBTRACTOR_BEHAVIORAL_SOURCE;
architecture Behavioral of
FULLSUBTRACTOR_BEHAVIORAL_SOURCE is
begin
process (A)
begin
if (A = "001" or A = "010" or A = "111") then
Y <= "11";
elsif (A = "011") then
Y <= "01";
elsif (A = "100") then
Y <= "10";
else
Y <= "00";
end if;
end process;
end Behavioral;
```

RTL Schematics:

\

Waveform

DEPARTMENT OF CSE(AIML)/CSBS

Day 7: Implementation of 4-bit Ripple Carry Adder and Carry Look-ahead Adder using VHDL (Behavioral Model)

NAME OF THE STUDENT	
ROLL NO.	
DATE OF EXPERIMENT	
DATE OF SUBMISSION	
GRADE/MARKS	
TEACHER'S SIGNATURE	

Assignment 1: Implementation of 4-bit Ripple Carry Adder in VHDL (Behavioral Model)

Theory: Ripple Carry Adders offer a straight forward approach to binary addition using cascaded full adders to gives the results of the addition of two n bit binary sequence. This adder includes cascaded full adders in its structure so, the carry will be generated ate very full adder stage in a ripple-carry adder circuit. These carry output at each full adder stage is forwarded to its next full adder and there applied as a carry input to it. This process continues up to its last full adder stage. So, each carry out put bit is rippled to the next stage of a full adder. By this reason, it is named as "Ripple Carry Adder".

Ripple Carry Adders serve as essential components in digital computers for performing binary addition. By employing basic logic gates like AND, OR, and XOR, these adders enable the addition of multi-bit binary numbers.

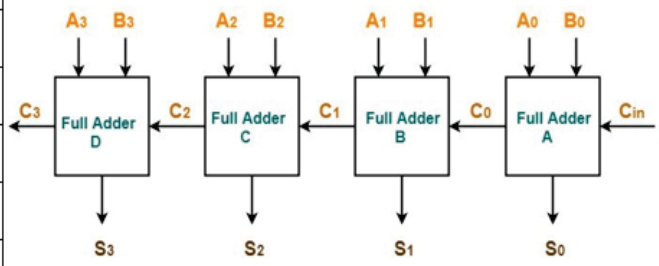
In a Ripple Carry Adder, each full adder waits for the carry bit from the previous full adder, thus propagating the carry bit through the chain. While they facilitate rapid design due to their simple layout, their performance is limited by carry propagation delays, resulting in relatively slower operation compared to alternative adder designs. The expressions for sum and carry for each stage is same as that of a full adder:

$Sum_i = A_i \oplus B_i \oplus C_{i-1}$

$Carry_i = A_iB_i + AC_{i-1} + BC_{i-1}$

Truth Table (Full Adder)

INPUT			OUTPUT	
A_i	B_i	C_{i-1}	Sum_i	$Carry_i$
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		



Behavioral Model of 4 bit Ripple Carry Adder:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity Ripple_Carry_Adder is
Port (
A, B : in STD_LOGIC_VECTOR(3 downto 0);
Cin  : in STD_LOGIC;
Sum  : out STD_LOGIC_VECTOR(3 downto 0);
Cout : out STD_LOGIC );
end Ripple_Carry_Adder;

architecture Behavioral of Ripple_Carry_Adder is
signal C : STD_LOGIC_VECTOR(3 downto 0);
begin
process(A, B, Cin)
begin
C(0) := Cin;
for i in 0 to 3 loop
Sum(i) <= A(i) XOR B(i) XOR C(i);
if (A(i) = '1' and B(i) = '1') or (A(i) = '1' and C(i) = '1') or (B(i) = '1' and C(i) = '1') then
C(i+1) <= '1';
else
C(i+1) <= '0';
end if;
end loop;
Cout <= C(4);
end process;
end Behavioral;
```

RTL Schematic:

Waveform

Assignment 2: Implementation of n-bit Carry Propagation Adder in VHDL (Structural Model)

Theory: In ripple carry adders, each adder block waits for the carry to arrive from its previous block. The i^{th} block waits for the $i\text{-}1^{th}$ block to produce its carry. This cause scarry propagation delay, which increases as the number of bits increase. Consider a 4-bit ripple carry adder. Even after inputs $A_3$and $B_3$are given, the 4^{th} full adder cannot produce steady state outputs until $C_3$is available at its final steady-state value.Similarly C_3 depends on C_2 and C_2 on C_1. Therefore, though the carry must propagate to all the stage s in order that out puts S_3 and C_4 settle at their final steady-state value.

The propagation time is equal to the propagation delay of each adder block, multiplied by the number of adder blocks in the circuit. For example, if each full adder stage has a propagation delay of 20 nano-seconds, then S_3 will reach its final correct value after 60(20×3) nanoseconds. The situation gets worse, if we extend the number of stages for adding more number of bits. Carry Lookahead Adders function by computing the carry signals in constant time. In this approach, two signals, known as Carry Propagator (P) and Carry Generator (G), are calculated for each stage of the adder.

The Carry Propagator signal indicates whether a carry will be propagated to the next stage from if the previous stage passed a carry signal. The Carry Generator signal indicates whether a carry will be generated at the current stage, regardless of input carry.

$P_i=A_i \oplus B_i$ & $G_i=A_i \cdot B_i$

Using the expressions for P and G, the sum & carry outputs for each stage are

$Sum_i= P_i \oplus C_i$ & $Carry_{i+1}= G_i+ (P_i \cdot C_i)$

By substituting the previous carry value(C_i) into the expression for C_{i+1}, we get:

$C_1= G_0+ P_0 \cdot C_0$

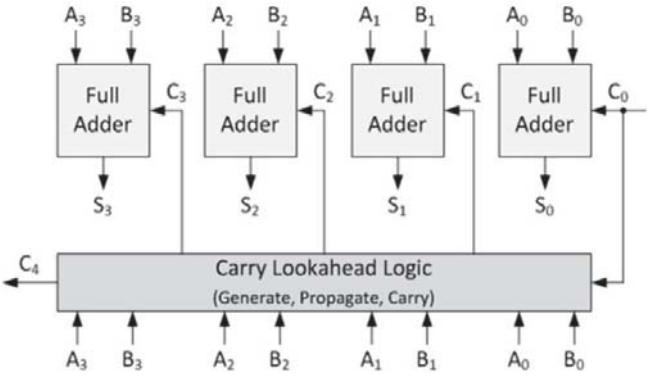
$C_2=G_1+P_1 \cdot C_1=G_1+ P_1 \cdot G_0+ P_1 \cdot P_0 \cdot C_0$

$C_3=G_2+P_2 \cdot C_2=G_2+P_2 \cdot G_1+P_2 \cdot P_1 \cdot G_0+ P_2 \cdot P_1 \cdot P_0 \cdot C_0$

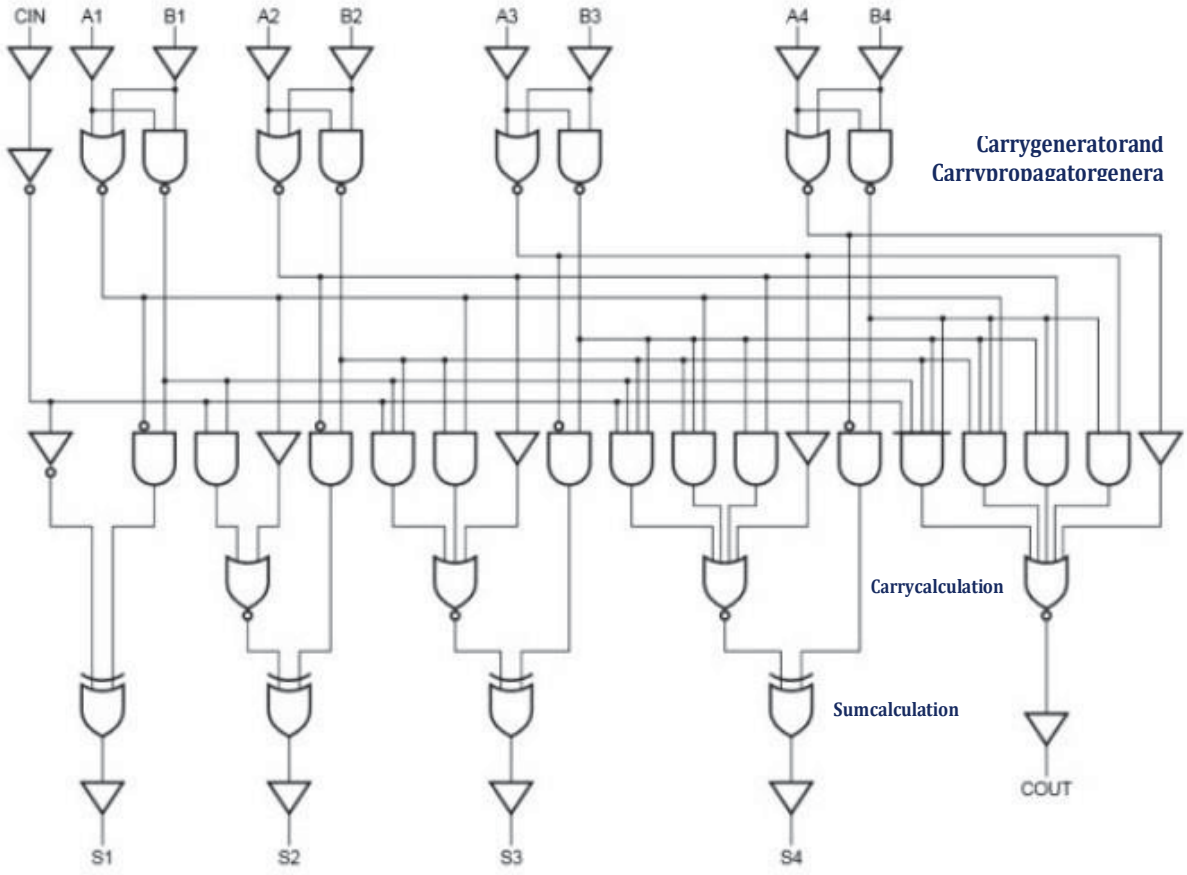
$C_4=G_3+ P_3 \cdot C_3=G_3+P_3 \cdot G_2 \cdot P_2 \cdot G_1+P_3 \cdot P_2 \cdot P_1 \cdot G_0+P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$

CLA can generate the entire output in 4 gate delays. 1 gate delay for generating the carry propagators and carry generators; 2 gate delays for the computing the carry bit and 1 additional gate delay for computing the sum bit.

INPUT			OUTPUT		
A_i	B_i	C_i	G_i	P_i	C_{i+1}
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			



Carry Generator & Carry Propagator



VHDL Code for Partial Full Adder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Partial_Full_Adder is
Port ( A : in STD_LOGIC;
B : in STD_LOGIC;
Cin : in STD_LOGIC;
S : out STD_LOGIC;
P : out STD_LOGIC;
G : out STD_LOGIC);
end Partial_Full_Adder;

architecture Behavioral of Partial_Full_Adder is

begin

S <= A xor B xor Cin;
P <= A xor B;
G <= A and B;

end Behavioral;
```

VHDL Code for Carry Look Ahead Adder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Carry_Look_Ahead is
Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
B : in STD_LOGIC_VECTOR (3 downto 0);
Cin : in STD_LOGIC;
S : out STD_LOGIC_VECTOR (3 downto 0);
Cout : out STD_LOGIC);
end Carry_Look_Ahead;

architecture Behavioral of Carry_Look_Ahead is

component Partial_Full_Adder
Port ( A : in STD_LOGIC;
B : in STD_LOGIC;
```

```
Cin : in STD_LOGIC;
S : out STD_LOGIC;
P : out STD_LOGIC;
G : out STD_LOGIC);
end component;

signal c1,c2,c3: STD_LOGIC;
signal P,G: STD_LOGIC_VECTOR(3 downto 0);
begin

PFA1: Partial_Full_Adder port map( A(0), B(0), Cin, S(0), P(0), G(0));
PFA2: Partial_Full_Adder port map( A(1), B(1), c1, S(1), P(1), G(1));
PFA3: Partial_Full_Adder port map( A(2), B(2), c2, S(2), P(2), G(2));
PFA4: Partial_Full_Adder port map( A(3), B(3), c3, S(3), P(3), G(3));
c1 <= G(0) OR (P(0) AND Cin);
c2 <= G(1) OR (P(1) AND G(0)) OR (P(1) AND P(0) AND Cin);
c3 <=
G(2) OR (P(2) AND G(1)) OR (P(2) AND P(1) AND G(0)) OR (P(2) AND P(1) AN
D P(0) AND Cin);
 Cout <=
 G(3) OR (P(3) AND G(2)) OR (P(3) AND P(2) AND G(1)) OR (P(3) AND P(2) AND
 P(1) AND G(0)) OR (P(3) AND P(2) AND P(1) AND P(0) AND Cin);

end Behavioral;
```

RTL Schematic:

Waveforms:

DEPARTMENT OF CSE(AIML)/CSBS

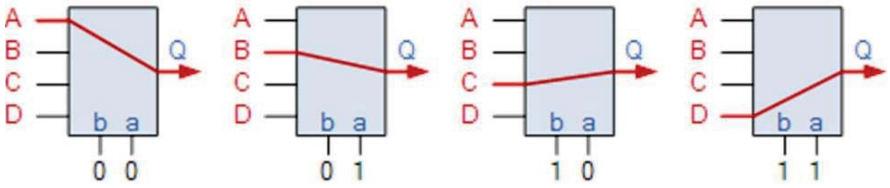
Day 8: Implementation of 4:1 MUX using 2:1 MUX (using Structural Method) and 4 Flip-flops in VHDL

NAME OF THE STUDENT	
ROLL NO.	
DATE OF EXPERIMENT	
DATE OF SUBMISSION	
GRADE/MARKS	
TEACHER'S SIGNATURE	

Assignment 1: Implementation of 4:1 MUX using 2:1 MUX in VHDL (Structural Model)

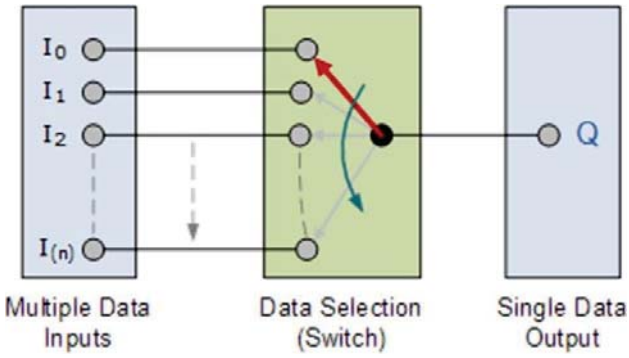
Theory:

Multiplexing is the generic term used to describe the operation of sending one or more analogue or digital signals over a common transmission line at different times or speeds and the device used to do that is called Multiplexer.



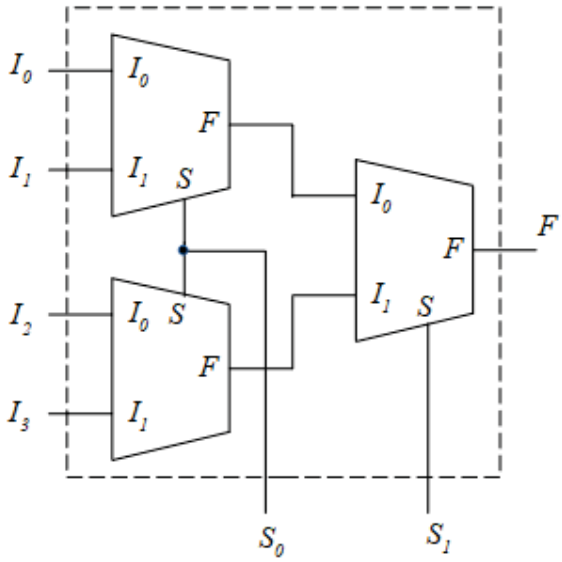
The multiplexer, shortened to "MUX" or "MPX", is a combinational logic circuit designed to switch one of several input lines through to a single common output line by the application of a control signal. Multiplexers operate like every fast-acting multiple position rotary switches connecting or controlling multiple input lines called "channels" one at a time to the output. Multiplexers, or MUX's, can be either digital circuits made from high-speed logic gates used to switch digital or binary data or they can be analogue types using transistors, MOSFET's or relays to switch one of the voltage or current in puts through to a single output.

Basic Multiplexing Switch



The rotary switch, also called a wafer switch as each layer of the switch is known as a wafer, is a mechanical device whose input is selected by rotating a shaft. In other words, the rotary switch is a manual switch that you can use to select individual data or signal lines simply by turning its inputs "ON" or "OFF". So how can we select each data input automatically using a digital device. In digital electronics, multiplexers are also known as data selectors because they can "select" each input line, are constructed from individual Analogue Switches encased in a single IC package as opposed to the "mechanical" type selectors such as normal conventional switches and relays. They are used as one method of reducing the number of logic gates required in a circuit design or when a single data line or data bus is required to carry two or more different digital signals. For example, a single 8-channel multiplexer, Generally, the selection of each input line in a multiplexer is controlled by an additional set of inputs called control lines and according to the binary condition of these control inputs, either "HIGH" or "LOW" the appropriate data input is connected directly to the output. Normally, a multiplexer has an even number of 2 n data input lines and a number of "control" inputs that correspond with the number of data

inputs.



VHDL code for 4 to 1 Mux

Behavioral Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity mux_4to1 is
 port(

    A,B,C,D : in STD_LOGIC;
    S0,S1: in STD_LOGIC;
    Z: out STD_LOGIC
  );
end mux_4to1;

architecture bhv of mux_4to1 is
begin
process (A,B,C,D,S0,S1) is
begin
  if (S0 ='0' and S1 = '0') then
     Z <= A;
  elsif (S0 ='1' and S1 = '0') then
     Z <= B;
  elsif (S0 ='0' and S1 = '1') then
```

```
    Z <= C;
  else
    Z <= D;
  end if;

end process;
end bhv;
```

Structural Model

VHDL Code for 2 to 1 Mux

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2_1 is
port(A,B : in STD_LOGIC;
S: in STD_LOGIC;
Z: out STD_LOGIC);
end mux2_1;

architecture Behavioral of mux2_1 is

begin

process (A,B,S) is
begin
if (S ='0') then
Z <= A;
else
Z <= B;
end if;
end process;

end Behavioral;
```

Structural Model for 4: 1 Mux

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux4_1 is
port(

A,B,C,D : in STD_LOGIC;
S0,S1: in STD_LOGIC;
Z: out STD_LOGIC
```

```
);
end mux4_1;

architecture Behavioral of mux4_1 is
component mux2_1
port( A,B : in STD_LOGIC;
S: in STD_LOGIC;
Z: out STD_LOGIC);
end component;
signal temp1, temp2: std_logic;

begin
m1: mux2_1 port map(A,B,S0,temp1);
m2: mux2_1 port map(C,D,S0,temp2);
m3: mux2_1 port map(temp1,temp2,S1,Z);

end Behavioral;
```

RTL Schematic:

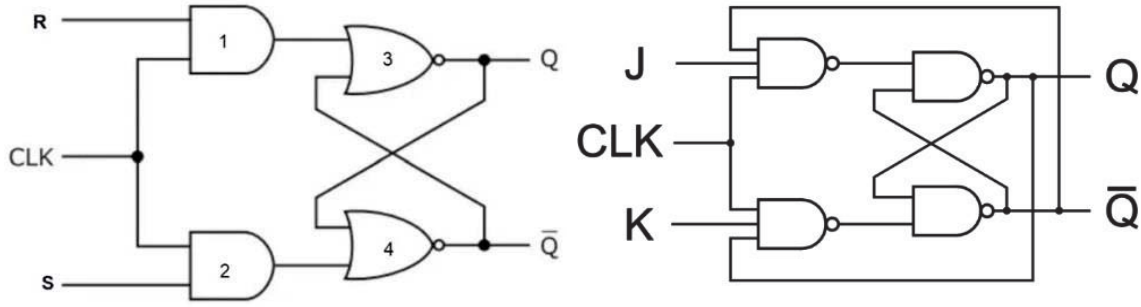
Waveform:

Assignment 2: Implementation of 4 Flip-flops (S-R, J-K, T and D)

Sequential circuits are digital circuits that store and use the previous state information to determine the in next state. Unlike combinational circuits, which only depend on the current input values to produce outputs, sequential circuits depend on both the current inputs and the previous state stored in memory elements.

SR flip-flop :

The SR flip-flop is one of the fundamental parts of the sequential circuit logic. SR flip-flop is a memory device and binary data of 1 bit can be stored in it. SR flip-flop has two stable states in which it can store data in the form of either binary zero or binary one. Like all flip-flops, an SR flip-flop is also an edge sensitive device. SR flip–flop is one of the most vital components in digital logic and it is also the most basic sequential circuit that is possible. The S (SET) input, sets the output to 1 and the R (RESET) input resets it to 0.

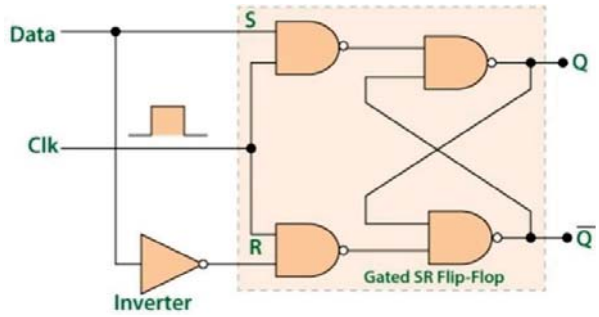


JK flip-flop:

JK flip-flop is a refinement of SR flip-flop where the indeterminate state of SR type is defined. Input J and K are respectively the set and reset inputs of the flip-flop. When both the inputs are high then the output of the flip-flop switches to its complemented state.

Dflip-flop:

To ensure that both S and R inputs don't become the same, we can use the Delay/ Data flip-flop. Here there is a single 'Data' input which is connected to the Set pin, while its complement goes to the Reset pin.



VHDL Code for S-R FlipFlop

```
library ieee;
use ieee. std_logic_1164.all;
use ieee. std_logic_arith.all;
use ieee. std_logic_unsigned.all;

entity SR_FF is
PORT( S,R,CLOCK: in std_logic;
Q, QBAR: out std_logic);
end SR_FF;

Architecture behavioral of SR_FF is
begin
PROCESS(CLOCK)
variable tmp: std_logic;
begin
if(CLOCK='1' and CLOCK'EVENT) then
if(S='0' and R='0')then
tmp:=tmp;
elsif(S='1' and R='1')then
tmp:='Z';
elsif(S='0' and R='1')then
tmp:='0';
else
tmp:='1';
end if;
end if;
Q <= tmp;
QBAR <= not tmp;
end PROCESS;
end behavioral;
```

RTL Schematic

VHDL Code for D FlipFlop

```
library ieee;
use ieee. std_logic_1164.all;
use ieee. std_logic_arith.all;
use ieee. std_logic_unsigned.all;

entity D_FF is
PORT( D,CLOCK: in std_logic;
Q: out std_logic);
end D_FF;

architecture behavioral of D_FF is
begin
process(CLOCK)
begin
if(CLOCK='1' and CLOCK'EVENT) then
Q <= D;
end if;
end process;
end behavioral;
```

RTL Schematic

VHDL Code for JK FlipFlop

```
library ieee;
use ieee. std_logic_1164.all;
use ieee. std_logic_arith.all;
use ieee. std_logic_unsigned.all;

entity JK_FF is
PORT( J,K,CLOCK: in std_logic;
Q, QB: out std_logic);
end JK_FF;

Architecture behavioral of JK_FF is
begin
PROCESS(CLOCK)
variable TMP: std_logic;
begin
if(CLOCK='1' and CLOCK'EVENT) then
if(J='0' and K='0')then
TMP:=TMP;
elsif(J='1' and K='1')then
TMP:= not TMP;
elsif(J='0' and K='1')then
TMP:='0';
else
TMP:='1';
end if;
end if;
Q<=TMP;
Q <=not TMP;
end PROCESS;
end behavioral;
```

RTL Schematic

VHDL Code for T FlipFlop

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity T_FF is
port( T: in std_logic;
Clock: in std_logic;
Q: out std_logic);
end T_FF;

architecture Behavioral of T_FF is
signal tmp: std_logic;
begin
process (Clock)
begin
if Clock'event and Clock='1' then

if T='0' then
tmp <= tmp;
elsif T='1' then
tmp <= not (tmp);
end if;
end if;
end process;
Q <= tmp;
end Behavioral;
```

RTL Schematic

Waveform:

DEPARTMENT OF CSE(AIML)/CSBS

Day 9: Implementation of Encoder and Decoder using VHDL

NAME OF THE STUDENT	
ROLL NO.	
DATE OF EXPERIMENT	
DATE OF SUBMISSION	
GRADE/MARKS	
TEACHER'S SIGNATURE	

Encoder:

An encoder is a digital circuit that converts a set of binary inputs into a unique binary code. The binary code represents the position of the input and is used to identify the specific input that is active. Encoders are commonly used in digital systems to convert a parallel set of inputs into a serial code.

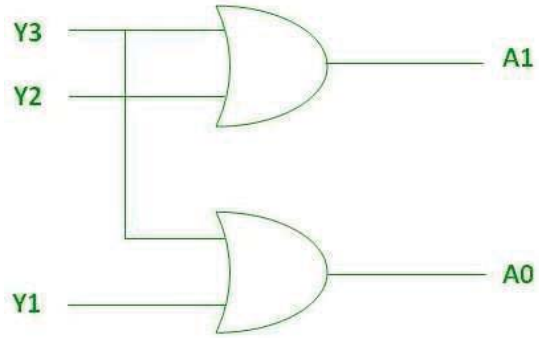
The basic principle of an encoder is to assign a unique binary code to each possible input. For example, a 2-to-4 line encoder has 2 input lines and 4 output lines and assigns a unique 4-bit binary code to each of the $2^2 = 4$ possible input combinations.

There are different types of encoders, including priority encoders, which assign a priority to each input, and binary-weighted encoders, which use a binaryweightingsystemtoassignbinarycodestoinputs.Encodersarewidely used in digital systems to convert parallel inputs into serial codes.

An Encoder has a maximum of2^n input lines and 'n' output lines, hence it encodes the information from 2^n inputs into an n-bit code. It will produce a binary code equivalent to the input, which is active High. Therefore, the encoder encodes 2^n input lines with 'n' bits.

Truth Table

INPUT				OUTPUT	
Y_3	Y_2	Y_1	Y_0	A_1	A_0
0	0	0	1		
0	0	1	0		
0	1	0	0		
1	0	0	0		



Logical expressions:

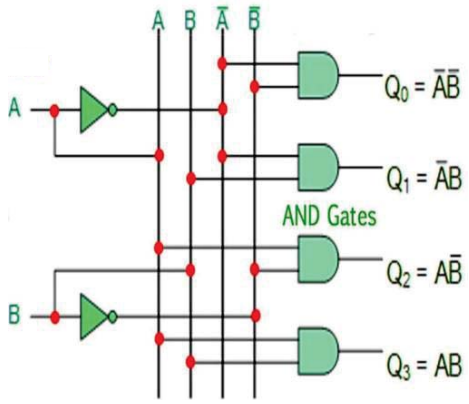
Decoder

A binary decoder is a digital circuit that converts a binary code into a set of outputs. The binary code represents the position of the desired output and is used to select the specific output that is active. Binary decoders are commonly used in digital systems to convert a serial code into a parallel set of outputs. The basic principle of a binary decoder is to assign a unique output to each possible binary code. For example, a binary decoder with 2 inputs and $2^2 = 4$ outputs can assign a unique output to each of the 4 possible 2-bit binary codes.

In Digital Electronics, discrete quantities of information are represented by binary codes. A binary code of n-bits is capable of representing upto 2^n distinct coded information. A decoder is a combinational circuit that converts binary information from n input lines to a maximum of 2^n unique output lines.

If a binary decoder receives n inputs it activates only one of its 2^n outputs based on that input with all other outputs deactivated. If the n-bit coded information has unused combinations, the decoder may have fewer than 2^n outputs. Binary decoders generate the 2^n (or fewer) minterms of n input variables with each input combination asserting a unique output.

INPUT			OUTPUT			
EN	S_1	S_0	O_3	O_2	O_1	O_0
0	X	X				
1	0	0				
1	0	1				
1	1	0				
1	1	1				



VHDL code for 4:2 Encoder Dataflow model

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity encoder2 is
port(
a : in STD_LOGIC_VECTOR(3 downto 0);
b : out STD_LOGIC_VECTOR(1 downto 0)
);
end encoder2;

architecture bhv of encoder2 is
begin

b(0) <= a(1) or a(2);
b(1) <= a(1) or a(3);

end bhv;
```

VHDL code for 4: 2 Encoder Behavioural Model

```
Library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity encoder is
port(
a : in STD_LOGIC_VECTOR(3 downto 0);
b : out STD_LOGIC_VECTOR(1 downto 0)
);
end encoder;

architecture bhv of encoder is
begin

process(a)
begin
case a is
when "1000" => b <= "00"; when "0100" => b <= "01"; when "0010" => b <=
"10"; when "0001" => b <= "11"; when others => b <= "ZZ";
end case;
end process;

end bhv;
```

VHDL code for 4: 2 Encoder using If-else statement

```
library IEEE;

use IEEE.STD_LOGIC_1164.all;

entity encoder1 is
port(
a : in STD_LOGIC_VECTOR(3 downto 0);
b : out STD_LOGIC_VECTOR(1 downto 0)
);
end encoder1;

architecture bhv of encoder1 is
begin

process(a)
begin
if (a="1000") then
b <= "00";
elsif (a="0100") then
b <= "01";
elsif (a="0010") then
b <= "10";
elsif (a="0001") then
b <= "11";
else
b <= "ZZ";
end if;
end process;

end bhv;
```

VHDL model for 2:4 decoder Dataflow model

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity decoder2 is
port( a : in STD_LOGIC_VECTOR(1 downto 0);
 b : out STD_LOGIC_VECTOR(3 downto 0) );
 end decoder2;
architecture bhv of decoder2 is
begin
b(0) <= not a(0) and not a(1);
b(1) <= not a(0) and a(1);
b(2) <= a(0) and not a(1);
b(3) <= a(0) and a(1);
end bhv;
```

VHDL model for 2:4 decoder Behavioural Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity decoder is
port( a : in STD_LOGIC_VECTOR(1 downto 0);
 b : out STD_LOGIC_VECTOR(3 downto 0) );
 end decoder;
architecture bhv of decoder is
begin
process(a)
begin
```

```
case a is when "00" => b <= "0001";
when "01" => b <= "0010";
when "10" => b <= "0100";
when "11" => b <= "1000";
end case;
end process;
end bhv;
```

VHDL model for 2:4 decoder using If-else statement

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity decoder1 is
port( a : in STD_LOGIC_VECTOR(1 downto 0);
b : out STD_LOGIC_VECTOR(3 downto 0) );
end decoder1;
architecture bhv of decoder1 is
begin
process(a)
begin
if (a="00") then b <= "0001";
elsif (a="01") then b <= "0010";
elsif (a="10") then b <= "0100";
else b <= "1000";
end if;
end process;
end bhv;
```

RTL Schematic for Encoder:

Waveform:

RTL Schematic for Decoder

Waveform:

DEPARTMENT OF CSE(AIML)/CSBS

Day 10: Implementation of Signed- Multiplier and Non-Restoring Division algorithm using VHDL

NAME OF THE STUDENT	
ROLL NO.	
DATE OF EXPERIMENT	
DATE OF SUBMISSION	
GRADE/MARKS	
TEACHER'S SIGNATURE	

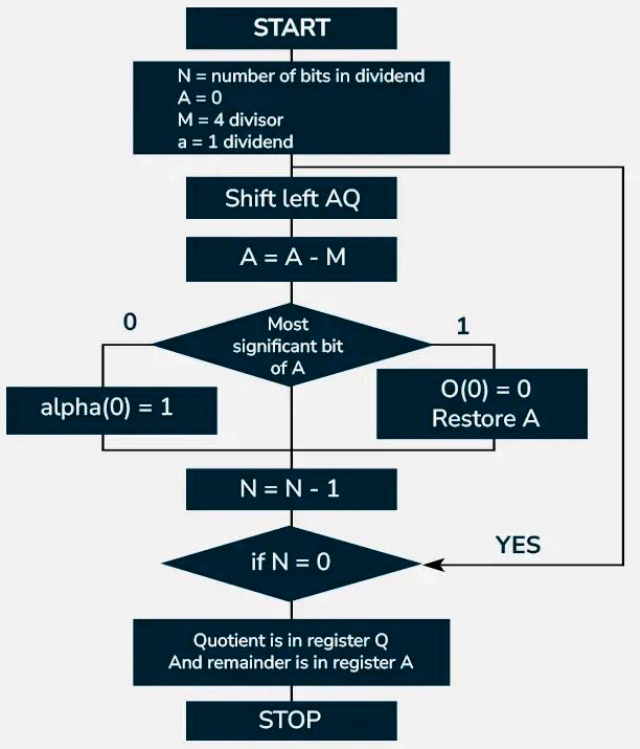
Assignment 1: Implementation of Signed Multiplier using VHDL

Theory: A signed multiplier is a digital circuit that performs multiplication on two's complement signed binary numbers. Unlike unsigned multipliers, signed multipliers must handle negative numbers correctly while maintaining arithmetic accuracy. One of the most efficient methods for signed multiplication is Booth's Algorithm, which reduces the number of partial products and optimizes hardware implementation.
For this, Two's Complement is used:

- Used to represent negative numbers in binary.
- The MSB (Most Significant Bit) indicates the sign:
  - 0 → Positive number
  - 1 → Negative number

Step	Operation	Description
1	Initialize	Load Multi cand (M), Multiplier (Q), a
2	Check Q_0Q_{-1}	If `01` → **A = A + M** If `10` → **A = A - M**
3	Arithmetic Right Shift	Shift A, Q, Q_{-1} right (preserves sign bit)
4	Repeat for N cycles	N = bit-width of multiplier
5	Final Result	Product = A & Q

Flow-chart of the Multiplication Algorithm



VHDL code for Signed- Multiplication Algorithm

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity BoothMultiplier is
   Port (
clk     : in  STD_LOGIC;
      start    : in  STD_LOGIC;
A_in    : in  STD_LOGIC_VECTOR(3 downto 0);  -- Multiplicand (signed)
B_in    : in  STD_LOGIC_VECTOR(3 downto 0);  -- Multiplier (signed)
      product  : out STD_LOGIC_VECTOR(7 downto 0);  -- 8-bit result
      ready    : out STD_LOGIC
   );
end BoothMultiplier;
```

Architecture:

```
architecture Behavioral of BoothMultiplier is
   type state_type is (IDLE, INIT, CHECK, ADD, SUB, SHIFT, DONE);
   signal state : state_type := IDLE;

   signal A_reg, Q_reg : STD_LOGIC_VECTOR(3 downto 0);
   signal M_reg       : STD_LOGIC_VECTOR(3 downto 0);
   signal Q_1         : STD_LOGIC := '0';  -- Q_-1 (previous bit)
   signal counter     : integer range 0 to 4;
   signal P_reg       : STD_LOGIC_VECTOR(7 downto 0);  -- Product register
begin
   process(clk)
      variable temp_A : STD_LOGIC_VECTOR(4 downto 0);  -- Extra bit for carry
   begin
      if rising_edge(clk) then
         case state is
            when IDLE =>
               ready <= '1';
               if start = '1' then
                  ready <= '0';
M_reg<= A_in;     -- Load Multiplicand
Q_reg<= B_in;     -- Load Multiplier
A_reg<= (others => '0');
                  Q_1 <= '0';
                  counter <= 4;
                  state <= INIT;
               end if;
```

```
            when INIT =>
               state <= CHECK;

            when CHECK =>
               if (Q_reg(0) = '1' and Q_1 = '0') then
                  state <= SUB;
elsif (Q_reg(0) = '0' and Q_1 = '1') then
                  state <= ADD;
               else
                  state <= SHIFT;
               end if;

            when ADD =>
temp_A := STD_LOGIC_VECTOR(unsigned('0' &A_reg) + unsigned('0' &M_reg));
A_reg<= temp_A(3 downto 0);
               state <= SHIFT;

            when SUB =>
temp_A := STD_LOGIC_VECTOR(unsigned('0' &A_reg) - unsigned('0' &M_reg));
A_reg<= temp_A(3 downto 0);
               state <= SHIFT;

            when SHIFT =>
               -- Arithmetic Right Shift (A & Q & Q-1)
               Q_1 <= Q_reg(0);
Q_reg<= A_reg(0) &Q_reg(3 downto 1);
A_reg<= A_reg(3) &A_reg(3 downto 1);  -- Preserves sign bit

               counter <= counter - 1;
               if counter = 0 then
                  state <= DONE;
               else
                  state <= CHECK;
               end if;

            when DONE =>
               product <= A_reg&Q_reg;  -- 8-bit product
               ready <= '1';
               state <= IDLE;
         end case;
      end if;
   end process;
end Behavioral;
```

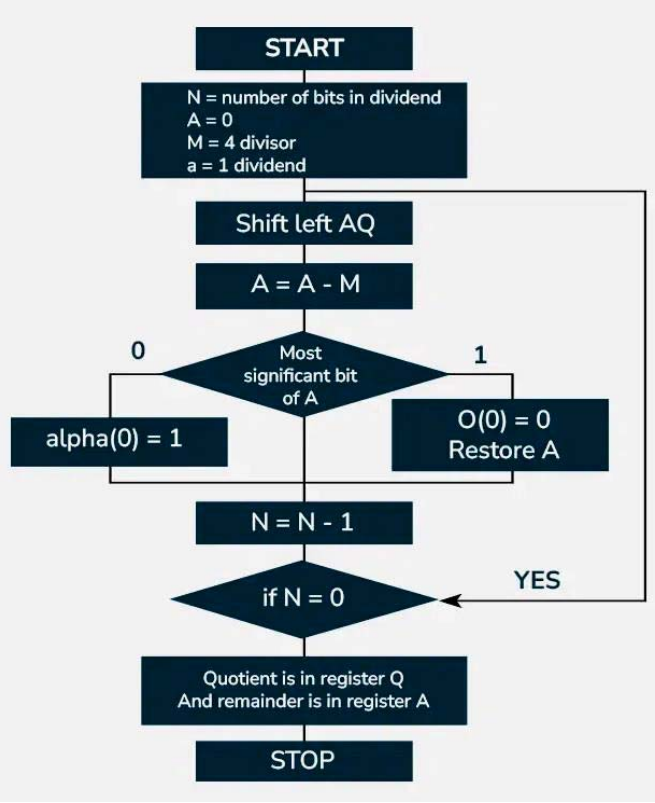
RTL Schematics:

Waveform:

Assignment 2: Implementation of Non-restoring Division Algorithm using VHDL

Theory: The **Non-Restoring Division Algorithm** is an efficient method for performing binary division in digital systems. Unlike the **Restoring Division Algorithm**, which requires restoring the partial remainder after a negative result, this algorithm avoids the restoration step by keeping track of the sign and adjusting the computation accordingly.

Flow-chart of the Non-Restoring Algorithm



VHDL code for Non-Restoring Algorithm

Architecture:

```
architecture Behavioral of NonRestoringDivider is
   type state_type is (IDLE, INIT, COMPARE, SHIFT, ADD_SUB, CORRECTION, DONE);
   signal state : state_type := IDLE;
   signal A_reg, Q_reg : std_logic_vector(N downto 0);  -- Extra bit for sign
   signal M_reg        : std_logic_vector(N downto 0);
```

```
    signal counter      : integer range 0 to N;
begin
    process(clk)
        variable temp_A : std_logic_vector(N downto 0);
    begin
        if rising_edge(clk) then
            case state is
                when IDLE =>
                    ready <= '1';
                    error <= '0';
                    if start = '1' then
                        if unsigned(divisor) = 0 then
                            error <= '1';
                            state <= DONE;
                        else
                            ready <= '0';
                            state <= INIT;
                        end if;
                    end if;
                when INIT =>
A_reg<= (others => '0');
Q_reg<= '0' & dividend;
M_reg<= '0' & divisor;
                    counter <= N;
                    state <= COMPARE;
                when COMPARE =>
                    if A_reg(N) = '1' then
                        state <= ADD_SUB;
                    else
                        state <= SHIFT;
                    end if;
                when SHIFT =>
                    -- Left shift AQ
A_reg<= A_reg(N-1 downto 0) &Q_reg(N);
Q_reg<= Q_reg(N-1 downto 0) & '0';
                    state <= ADD_SUB;
                when ADD_SUB =>
                    if A_reg(N) = '1' then
                        -- A is negative, add M
A_reg<= std_logic_vector(unsigned(A_reg) + unsigned(M_reg));
                    else
                        -- A is positive, subtract M
A_reg<= std_logic_vector(unsigned(A_reg) - unsigned(M_reg));
                    end if;

                    -- Set quotient bit
Q_reg(0) <= not A_reg(N);

                    counter <= counter - 1;
```

```
                if counter = 0 then
                  state <= CORRECTION;
                else
                  state <= COMPARE;
                end if;

            when CORRECTION =>
                if A_reg(N) = '1' then
                  -- Final remainder is negative, add M back
A_reg<= std_logic_vector(unsigned(A_reg) + unsigned(M_reg));
                end if;
                state <= DONE;

            when DONE =>
                quotient <= Q_reg(N-1 downto 0);
                remainder <= A_reg(N-1 downto 0);
                ready <= '1';
                state <= IDLE;
          end case;
      end if;
   end process;
end Behavioral;
```

RTL Schematics:

Waveform: